

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE GOIÁS
ESCOLA POLITÉCNICA E DE ARTES
GRADUAÇÃO EM CIÊNCIAS DA COMPUTAÇÃO



Mini Sistema Operacional de Controle de Alarme Residencial

CARLOS EDUARDO WATANABE NUNES, LUCAS BARBOSA ANTONELLI, LUCAS
MEDOLLA DE PAULA

GOIÂNIA
2025

1. DESCRIÇÃO DO SISTEMA ESCOLHIDO

1.1 Visão Geral

O sistema desenvolvido consiste em um Mini Sistema Operacional em espaço de usuário para controle de alarme residencial, implementado em linguagem C para ambiente Linux. O sistema simula um alarme doméstico completo com sensores de porta, janela e detecção de movimento, oferecendo funcionalidades de armamento, desarmamento e monitoramento contínuo.

1.2 Funcionalidades Principais O sistema oferece as seguintes funcionalidades:

- Armamento e Desarmamento por Senha: Sistema de autenticação numérica configurável.
- Simulação de 3 Sensores Independentes:
 - Sensor 0: Porta (detecta abertura/fechamento).
 - Sensor 1: Janela (detecta abertura/fechamento).
 - Sensor 2: Movimento (detecta presença).
- Sirene Lógica: Alerta visual e auditivo quando o alarme é disparado.
- Logging Persistente: Registro automático de todos os eventos com timestamps (carimbos de data/hora).
- Interface Interativa: Controle via linha de comando com feedback em tempo real.
- Configuração Externa: Parâmetros ajustáveis via arquivo de configuração.

1.3 Estados do Sistema O alarme opera em três estados distintos:

1. DISARMED (Desarmado): Estado inicial; sensores são monitorados, mas não disparam o alarme.
2. ARMED (Armado): Sistema ativo; qualquer acionamento de sensor dispara o alarme.
3. TRIGGERED (Disparado): Alarme ativado; sirene tocando continuamente até o desarmamento.

2. ARQUITETURA DO MINI SISTEMA OPERACIONAL

2.1 Modelo de Tarefas Concorrentes

O sistema foi projetado utilizando o modelo de múltiplas threads (POSIX threads) executando concorrentemente. Três threads principais compõem o núcleo do sistema:

Thread 1: Interface (UI)

- Tipo: Tarefa reativa a eventos.
- Função: `interface_task()`
- Responsabilidade: Receber comandos do usuário via stdin e enfileirá-los para processamento.
- Ciclo de Vida: Executa em loop até receber o comando exit.

Thread 2: Core (Processamento)

- Tipo: Tarefa reativa a eventos.
- Função: `core_task()`
- Responsabilidade: Consumir eventos da fila FIFO, processar comandos e atualizar o estado do sistema.
- Ciclo de Vida: Executa em espera passiva (condition variable) até receber eventos.

Thread 3: Monitor (Vigilância)

- Tipo: Tarefa periódica.
- Função: `monitor_task()`
- Responsabilidade: Verificar periodicamente o estado do alarme e acionar a sirene quando necessário.
- Ciclo de Vida: Executa a cada `config_sensor_interval` segundos usando `sleep()`.

2.2 Diagrama de Arquitetura

O diagrama abaixo representa a arquitetura do projeto, e como toda a aplicação é organizada.

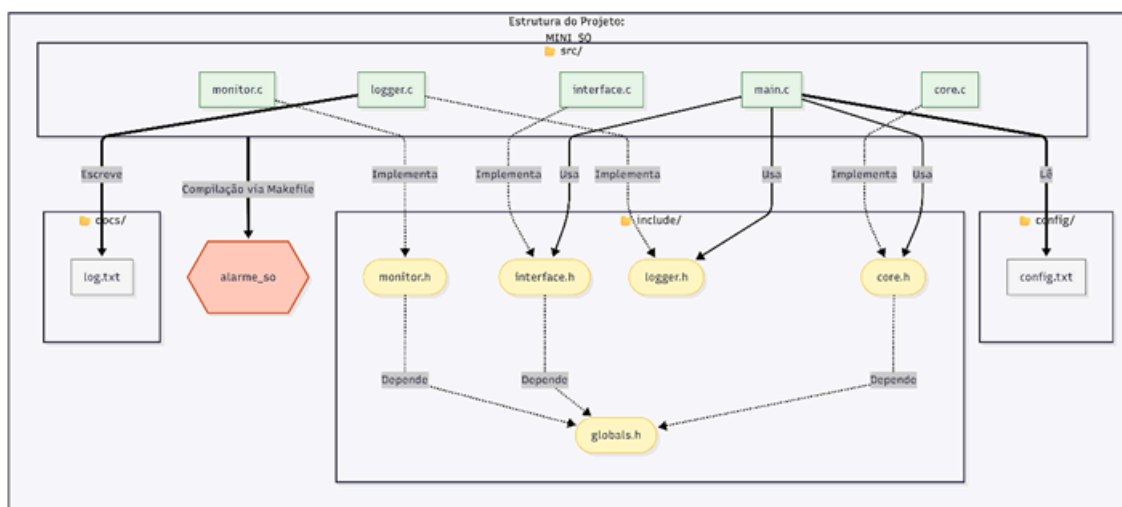


Figura 1 – Organização da Aplicação

A figura abaixo representa a comunicação de toda a aplicação e como ela se divide na memória.

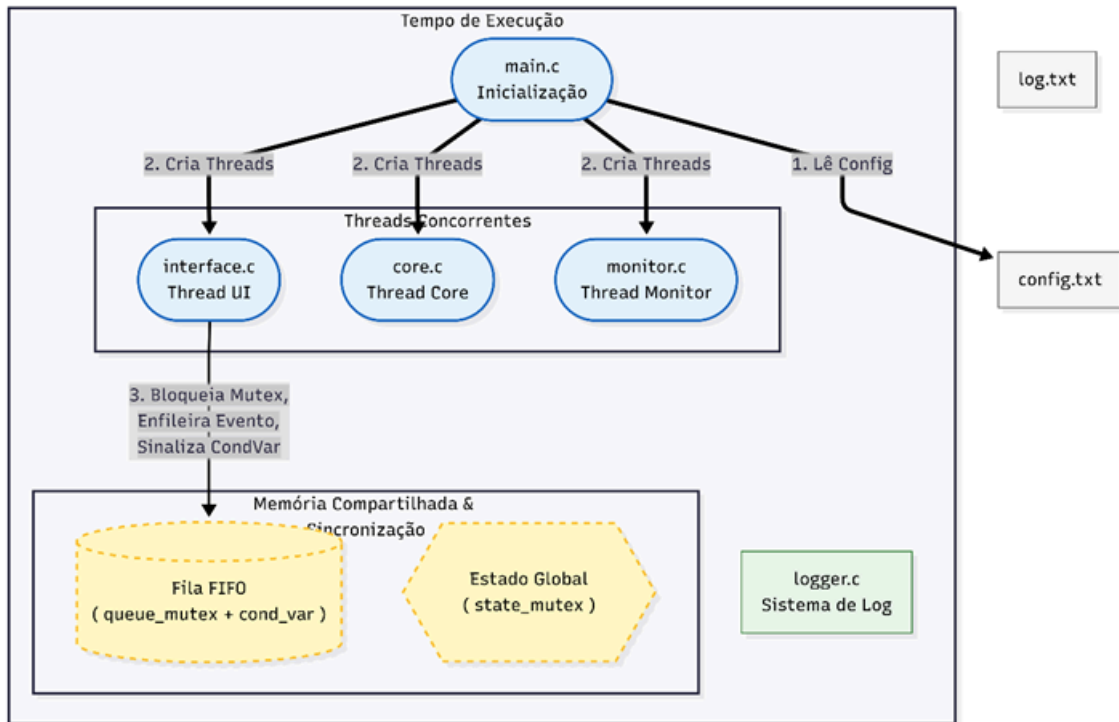


Figura 2 – Comunicação da Aplicação

2.3 Comunicação entre Tarefas

Fila de Eventos FIFO Circular Implementamos uma fila circular em memória compartilhada como principal mecanismo de comunicação:

- Capacidade: 10 eventos (definido por EVENT_QUEUE_SIZE).
- Estrutura: Array circular com ponteiros head (início) e tail (fim).
- Tipos de Eventos:
 - EVT_CMD_ARM: Comando para armar o sistema.
 - EVT_CMD_DISARM: Comando para desarmar o sistema.
 - EVT_TOGGLE_SENSOR: Simulação de acionamento de sensor.
 - EVT_STOP: Sinalização de encerramento.

Memória Compartilhada As threads compartilham as seguintes estruturas de dados:

- current_state: Estado atual do alarme (DISARMED/ARMED/TRIGGERED).
- sensors[3]: Array com o estado de cada sensor (0=inativo, 1=ativo).
- event_queue[]: Fila de eventos pendentes.
- queue_count, queue_head, queue_tail: Controle da fila.

2.4 Sincronização

Mutexes Implementados

1. `state_mutex`: Protege `current_state` e `sensors[]`. Usado em toda leitura/escrita do estado do sistema para evitar inconsistências.
2. `log_mutex`: Protege a escrita no arquivo `log.txt`. Usado em toda chamada à função `log_msg()` para evitar corrupção de dados.
3. `queue_mutex`: Protege `event_queue[]` e seus contadores. Usado no enfileiramento e desenfileiramento para evitar perda ou sobrescrita de eventos.

Condition Variable: A variável `queue_cond` é usada para notificação eficiente entre threads. A Thread de Interface sinaliza (`pthread_cond_signal`) ao adicionar um evento, enquanto a Thread Core aguarda (`pthread_cond_wait`) quando a fila está vazia. Isso evita busy-waiting (espera ocupada), reduzindo o uso de CPU.

Padrão Produtor-Consumidor: A comunicação segue o padrão clássico:

Exemplo de código (Pseudocódigo): Produtor (Interface): Bloqueia mutex -> Adiciona evento -> Sinaliza cond -> Desbloqueia mutex. Consumidor (Core): Bloqueia mutex -> Se fila vazia, espera cond -> Remove evento -> Desbloqueia mutex.

2.5 Escalonamento

O sistema implementa um escalonamento cooperativo simplificado:

- Thread Interface: Bloqueante em `scanf()`, libera a CPU aguardando entrada.
- Thread Core: Espera passiva com condition variable, libera a CPU quando a fila está vazia.
- Thread Monitor: Usa `sleep()` para periodicidade, liberando a CPU durante o intervalo.
- Finalização Coordenada: A flag `system_running` permite o término gracioso de todas as threads.

3. DESCRIÇÃO DOS ARQUIVOS

3.1 Código-Fonte (Módulos .c/.h)

- `main.c`: Ponto de entrada. Inicializa variáveis, carrega configurações, cria as threads e aguarda o término.
- `globals.h`: Declarações de tipos, enums e variáveis globais compartilhadas.
- `core.c` / `core.h`: Processamento de eventos e lógica de negócio (validação de senha, mudança de estados).
- `interface.c` / `interface.h`: Interface com o usuário (leitura de comandos) e envio de eventos para a fila.
- `monitor.c` / `monitor.h`: Monitoramento periódico. Verifica se o estado é `TRIGGERED` e exibe alertas.
- `logger.c` / `logger.h`: Sistema de logging persistente e thread-safe com timestamps.

3.2 Arquivo de Configuração (config.txt)

Formato simples chave=valor.

Exemplo:

- INTERVAL=3
- PASSWORD=1234

3.3 Arquivo de Log (log.txt)

Registra inicialização, comandos, mudanças de sensores, disparos e erros.

Exemplo de saída: [Fri Dec 12 00:10:55 2025] ALERTA: ALARME DISPARADO POR SENSOR!

4. PRINCIPAIS DECISÕES DE PROJETO

- **Uso de Threads vs Processos:** Optamos por threads (pthreads) em vez de processos separados devido ao compartilhamento de memória mais eficiente, sincronização mais simples com mutexes e menor overhead de troca de contexto.
- **Fila Circular vs Pipe:** Implementamos uma fila circular em memória compartilhada para ter controle total sobre o comportamento da fila e permitir a integração natural com condition variables, sem a necessidade de chamadas de sistema (system calls) excessivas.
- **Variável de Condição para Espera:** A Thread Core utiliza pthread_cond_wait() para eliminar a espera ocupada (busy-waiting), garantindo economia de CPU e resposta imediata aos eventos.
- **Arquivo de Log com Flush Imediato:** Utilizamos fflush() após cada escrita no log para garantir a persistência dos dados mesmo em caso de falha crítica do programa.
- **Simulação de Sensores:** Os sensores são controlados por comandos do usuário em vez de hardware real para garantir a portabilidade do código e focar nos conceitos de Sistemas Operacionais (concorrência e sincronização).

5. DIFICULDADES ENCONTRADAS E SOLUÇÕES

1. **Condição de Corrida no Estado:** Inicialmente, leituras inconsistentes ocorriam. Solucionado com a implementação do state_mutex protegendo todas as operações de estado.
2. **Deadlock na Finalização:** A Thread Core ficava bloqueada esperando eventos ao sair. Solucionado com o uso de pthread_cond_broadcast() e verificação da flag system_running.
3. **Mensagens de Log Intercaladas:** Escritas concorrentes corrompiam o log. Solucionado com o log_mutex.
4. **Fila de Eventos Cheia:** Eventos eram descartados em testes rápidos. Aumentamos a fila e implementamos verificações de limite.

6. POSSÍVEIS MELHORIAS E TRABALHOS FUTUROS

- Simulação de Ruído e Debounce: Adicionar realismo aos sensores simulando falhas intermitentes.
- Prioridade de Eventos: Processar eventos críticos (como incêndio) antes de logs ou comandos simples.
- Watchdog Timer: Implementar um mecanismo para reiniciar threads travadas automaticamente.
- Integração com Hardware: Adaptar o sistema para ler GPIOs reais de uma Raspberry Pi ou Arduino.

7. CONCEITOS DE SISTEMAS OPERACIONAIS APLICADOS

O projeto aplicou na prática os seguintes conceitos:

- Processos e Threads: Criação, join e compartilhamento de memória.
- Programação Concorrente: Execução paralela e comunicação assíncrona.
- Sincronização: Mutexes (exclusão mútua) e Condition Variables.
- IPC (Inter-Process Communication): Fila de mensagens e memória compartilhada.
- Escalonamento: Uso cooperativo de CPU e controle de tempo (sleep).

8. CONCLUSÃO

O projeto do Mini Sistema Operacional de Controle de Alarme Residencial atingiu todos os objetivos propostos, implementando com sucesso os conceitos fundamentais de Sistemas Operacionais II.

O sistema demonstra um exemplo prático e funcional de controle em tempo real. A arquitetura modular facilita a manutenção, e as funcionalidades extras (como a fila circular thread-safe e o logging persistente, e demonstram um domínio avançado dos conceitos de sincronização. O código é robusto, funcional e serve como uma base sólida para estudos de sistemas concorrentes.